



OPEN Deploying TinyML for energy-efficient object detection and communication in low-power edge AI systems

Ch Madhu Bhushan¹, Priya Koppuravuri¹, Nomitha Prasanthi¹, Firoj Gazi¹, Md Muzakkir Hussain¹, Mohammad Abdussami¹, Aguru Aswani Devi¹ & Jamilurrahman Faizi²✉

Edge Artificial Intelligence (Edge AI) is driving the widespread deployment of neural network models on resource-constrained microcontroller units (MCUs), enabling real-time, on-device data processing. This approach significantly reduces cloud dependency, making it ideal for applications in industrial automation and IoT. However, the deployment of deep learning models on such constrained devices poses significant challenges due to limitations in memory, computational power, and energy capacity. This paper presents a real-time object detection system optimized for energy efficiency and scalability, which integrates well-established model compression techniques, such as quantization, with a low-cost MCU-based platform. The system leverages MobileNetV2, a lightweight neural network, quantized to achieve the best trade-offs between accuracy and resource consumption. The proposed solution integrates a camera and Wi-Fi module for capturing and transmitting image data, utilizing dual-mode TCP/UDP communication to balance reliability and low-latency transmission for IoT applications. We present a comprehensive system-level analysis, exploring the trade-offs between latency, memory, energy consumption, and model size. The Visual Wake Words (VWW) dataset is used for this research, which demonstrates the practical performance and scalability of the system for real-time applications in smart devices, industrial monitoring, and environmental sensing. This work emphasizes the integration of TinyML models with constrained hardware and offers a foundation for scalable, autonomous, energy-efficient Edge AI solutions. Quantitatively, 8-bit post-training quantization achieved $3\text{--}4\times$ storage reduction, yielding deployable flash footprints of 286–536 KB within a 1 MB flash / 256 KB SRAM budget, on-device inference latency ranged from 3.47 to 14.98 ms per frame with energy per inference of 10.6–22.1 J, while quantized MobileNet variants maintained accuracy ≥ 0.85 . In wireless reporting, UDP reduced one-way latency relative to TCP, whereas TCP provided higher delivery reliability, underscoring application-dependent protocol trade-offs for real-time embedded deployments.

Edge Artificial Intelligence (Edge AI)^{1,2} has emerged as a transformative paradigm, enabling real-time data processing directly on resource-constrained devices such as microcontroller units (MCUs). By performing inference locally on-device, Edge AI significantly reduces reliance on cloud-based computation, which leads to lower latency, optimized bandwidth utilization, and enhanced data privacy. These characteristics are particularly critical in applications requiring timely decisions and minimal energy consumption, including industrial automation, smart healthcare, wearable devices^{3–5}, and Internet of Things (IoT) networks. Despite these advantages, deploying deep learning models on low-power MCUs presents substantial challenges, as conventional neural networks often exceed the memory, computational, and energy capabilities of these platforms. Without careful design and optimization, deploying state-of-the-art models on MCUs can result in infeasible memory requirements, prolonged inference times, and increased power consumption, undermining the practical viability of Edge AI solutions.

¹Department of CSE, SRM University AP, Amaravati, Andhra Pradesh, India. ²Faculty of Computer science, Nangarhar University, Jalalabad, Afghanistan. ✉email: Jamilurrahmanfaizi@gmail.com

To overcome these constraints, model compression techniques^{6,7} have become essential. Quantization reduces the numerical precision of model weights from 32-bit floating-point to lower-bit representations, typically 8-bit integers, thereby decreasing memory footprint, computational cost, and inference latency while maintaining acceptable accuracy levels. Pruning eliminates redundant weights and network connections, further reducing resource consumption and improving inference efficiency. When combined with lightweight neural network architectures such as MobileNetV2^{8,9} and other lightweight models^{10,11}, which employs depth-wise separable convolutions to reduce the total number of parameters, these strategies allow real-time vision models to be executed efficiently on embedded devices. In addition, transfer learning with pre-trained models, such as ImageNet-initialized MobileNetV2, enables rapid adaptation to target tasks with minimal computational overhead, reducing training time and improving generalization in resource-constrained environments. For ease of reference, a comprehensive list of abbreviations used throughout this paper is provided in Table 1.

In this work, we propose a comprehensive, end-to-end framework for deploying real-time object detection on low-power MCUs. The system integrates a quantized^{12–14} MobileNetV2 model with a low-resolution camera directly connected to the MCU, enabling on-device inference of captured images. Detection results are transmitted to a host system via dual-mode TCP/UDP communication^{15,16}, providing flexibility to prioritize either reliability or low latency depending on the application requirements, and aligning with collaborative edge–host inference paradigms¹⁷. The framework is evaluated using the Visual Wake Words (VWW) dataset (<https://github.com/Futuristic-Ubiquitous-Networking-Lab/VWW>), a widely recognized benchmark for binary human presence detection in images. To facilitate practical deployment on MCUs, a carefully selected subset of the dataset is used, maintaining strict class balance across training, validation, and test splits. Input image resolution and network width multiplier are systematically varied to explore the trade-offs among accuracy, memory usage, inference latency, and energy consumption, providing a detailed performance characterization under different deployment configurations.

It is important to clarify that the focus of this study is on the practical evaluation of established TinyML techniques rather than introducing new neural network architectures or compression algorithms. While methods such as quantization, pruning, and knowledge distillation^{18–20} are applied, the primary contribution lies in the systematic analysis of these approaches in a real-world MCU deployment scenario. We provide precise measurements of model size, memory utilization, energy consumption, inference latency, and communication efficiency, highlighting both the effectiveness and limitations of each strategy. Notably, pruning and knowledge distillation demonstrate limited benefits under stringent memory and energy constraints, whereas quantization consistently enables feasible deployment with minimal accuracy loss. By emphasizing reproducibility and detailed performance characterization, this study offers valuable insights for practitioners designing Edge AI systems on low-power devices.

Abbreviation	Full form
ARM	Advanced RISC Machine
AutoML	Automated Machine Learning
CNN	Convolutional Neural Network
CMSIS-NN	Cortex Microcontroller Software Interface Standard – Neural Network
Edge AI	Edge Artificial Intelligence
IoT	Internet of Things
JSON	JavaScript Object Notation
KB	Kilobyte
KD	Knowledge Distillation
LOS	Line-of-Sight
MB	Megabyte
MCU	Microcontroller Unit
MV1	MobileNet Version 1
MV2	MobileNet Version 2
NAS	Neural Architecture Search
NQ	Non-Quantized (model)
Q	Quantized (model)
RGB	Red, Green, Blue
SCNN	Simple Convolutional Neural Network
SRAM	Static Random-Access Memory
TCP	Transmission Control Protocol
TFLM	TensorFlow Lite Micro
UDP	User Datagram Protocol
VWW	Visual Wake Words
Wi-Fi	Wireless Fidelity

Table 1. Abbreviations.

While this work demonstrates practical deployment on MCUs, it does not perform exhaustive benchmarking against all state-of-the-art TinyML approaches, such as quantization-aware training (QAT), MobileNetV3^{21,22}, or MCUNet architectures. The intent is to provide a reproducible, scalable workflow for energy-efficient object detection on resource-constrained platforms, offering clear metrics and insights into achievable performance and existing limitations. By presenting comprehensive, empirically validated trade-offs between accuracy, memory, energy, and latency, this study equips researchers and practitioners with actionable guidance for deploying TinyML solutions in real-world IoT, smart home, and industrial monitoring applications, bridging the gap between theoretical model optimization and practical embedded deployment. **To summarise, the key highlights of our work is as follows:**

Contributions

- We present a practical deployment framework for real-time object detection on low-power MCUs, integrating quantized MobileNetV2 with adaptive TCP/UDP communication for efficient data transmission.
- We systematically evaluate the trade-offs between accuracy, latency, energy consumption, and memory footprint on real hardware using the VWW dataset, highlighting the impact of model compression techniques.
- We provide insights into the limitations of pruning and knowledge distillation in highly constrained environments, demonstrating that quantization is the most effective strategy for MCU deployment.
- We offer a reproducible methodology for researchers and practitioners to deploy energy-efficient deep learning models on embedded devices, bridging the gap between theoretical model optimization and practical TinyML implementation.

Paper organization

The remainder of this paper is organized as follows. Section “Related work” presents a review of related work in the field of TinyML, model compression, and real-time object detection on embedded systems, highlighting key advancements and challenges. In Sect. “Methods”, we provide a detailed description of the system architecture, including the hardware setup, model design, and the deployment process used to enable real-time object detection on resource-constrained devices. Section “Results” discusses the experimental methodology, covering the dataset details, hardware specifications, and the training process employed for fine-tuning the model. Section “Discussion” presents the results of our experiments, where we evaluate model size, inference latency, energy consumption, and communication performance over TCP and UDP. In Sect. “Conclusion”, we offer a discussion of the results, highlighting the trade-offs and challenges encountered in deploying deep learning models on constrained devices, and the lessons learned from our approach. Finally, Sect. “Future work” concludes the paper and outlines potential directions for future work in the areas of model optimization, hardware acceleration, and the broader deployment of TinyML models in IoT and Edge AI applications.

Related work

The increasing adoption of microcontroller units for machine learning applications has driven advancements in hardware capable of running neural network models, with numerous studies exploring various microcontroller units, such as the FRDM-K64F²³ and FS32K142WAT0WLFT^{24,25}. A primary challenge in deploying deep learning models on these microcontroller units is the limited flash memory and SRAM, which directly impact both the feasibility and performance of the models²⁶. In this research, we utilize a microcontroller unit that is compatible with TensorFlow Lite, enabling optimized inference pipelines that are specifically tailored for resource-constrained environments. The Visual Wake Words (VWW) task, which is designed to detect human presence in images, requires a minimum of 250 KB of SRAM for effective operation²⁷, making the selected microcontroller unit with 256 KB of SRAM and 1 MB of flash memory a suitable choice for deployment.

Previous studies have evaluated MobileNet architectures, including MobileNetV1^{9,28,29} and MobileNetV2, on various datasets such as ImageNet^{30,31} and VWW²⁷, but there is a gap in the application of transfer learning on microcontroller units. Transfer learning, which leverages pre-trained models for fine-tuning on specific tasks, has gained popularity for improving generalization, reducing training time, and minimizing computational overhead. In this study, MobileNet models pre-trained on ImageNet are fine-tuned for the VWW task, optimizing inference efficiency and accuracy for constrained hardware environments.

Additionally, TCP/UDP-based image transmission has significantly improved data communication in Edge AI systems. TCP ensures reliable data delivery, which is critical for accuracy-dependent tasks, while UDP reduces latency, making it ideal for time-sensitive applications³². While deep learning model compression has been extensively studied, few studies systematically compare the energy consumption of MobileNet models on microcontroller units versus general-purpose computers, which is crucial for battery-powered applications^{33,34}. This research fills that gap by evaluating the trade-offs between latency, accuracy, and energy consumption across different platforms.

Furthermore, the integration of TCP/UDP for image transmission introduces challenges that require a balance between efficiency and reliability. By addressing these challenges, our work contributes to the scalable deployment of deep learning models on microcontroller units, demonstrating their viability in Edge AI and IoT applications.

A summary of the related work is provided in Table 2, which highlights key studies on optimization, TinyML, and hardware used in deploying deep learning models for embedded systems.

Citation	Use Case	Methodology	Hardware Used
Lokhande et al. ³⁵ , 2025	Real-time object detection in video surveillance on edge devices.	MobileNetV2-based deep learning inference optimized for resource-constrained environments.	Low-power microcontroller-based edge devices with cameras for on-device processing.
Jaiswal et al. ³⁶ , 2025	Real-time multiple object detection using TinyML approach.	Tiny-ML-based object detection models optimized for real-time inference on embedded devices.	Raspberry Pi microcontroller-based edge devices for embedded inference.
Stojanović et al. ³⁷ , 2025	Resource-aware object detection and recognition across the edge-fog computing continuum.	Edge AI-based object detection and recognition optimized for resource efficiency and deployment across edge and fog nodes.	Edge devices and fog computing nodes for real-time embedded inference.
El Zeinaty et al. ³⁸ , 2025	Design and evaluation of object detection models for TinyML applications.	Comparative analysis of TinyML object detection models, discussing challenges, optimizations, and emerging solutions for resource-constrained devices.	Generic MCUs and edge devices used in TinyML deployments.
Novac et al. ³⁹ , 2023	Deploying deep neural networks for AI inference on resource-constrained microcontrollers.	Quantization of deep neural networks to reduce model size and computational requirements for MCU deployment.	MCUs with limited memory and processing power.
Liberis et al. ⁴⁰ , 2023	Enabling smart applications on microcontrollers through differentiable neural network pruning.	Differentiable neural network pruning to reduce model size and computation while maintaining performance.	MCUs for on-device AI inference in wearable and IoT applications.
Moosmann et al. ³² , 2023	Deploying low-memory, low-power object detection models on MCUs.	Quantized TinyML object detection network (TinyISSMO-YOLO) optimized for low-memory and low-power inference on MCUs.	Low-power microcontrollers for embedded real-time inference.
Dai ³⁴ , 2020	Real-time and accurate object detection on edge devices.	Deployment of deep learning models using TensorFlow Lite for efficient on-device inference.	Edge devices/MCUs for real-time embedded inference.
Warden et al. ⁴¹ , 2019	Implementing machine learning on MCUs.	Deployment of TensorFlow Lite models on low-power MCUs for TinyML applications.	Arduino and other ultra low power microcontrollers.
Our work	Optimizing lightweight CNNs for embedded vision.	Explores quantization and width multiplier scaling for efficient deployment of CNNs.	ARM Cortex-M and Cortex-M7 based microcontrollers.

Table 2. Summary of Selected Papers on Optimization, TinyML, and Hardware Used.

Methods

This section outlines the experimental approach and procedures used to evaluate the proposed energy-efficient object detection system deployed on low-power microcontroller units (MCUs). It includes a detailed description of the dataset, hardware specifications, model architecture, and optimization techniques.

Dataset

The Visual Wake Words (VWW) dataset is a widely recognized benchmark for vision-based tasks in resource-constrained IoT applications, such as surveillance, smart homes, and wearable devices. The dataset consists of approximately 115,000 training images and 8,000 validation images, labeled as either “human present” or “no human present.” The VWW dataset is adapted from the COCO dataset and is structured using JSON files for binary classification, ensuring consistency and reproducibility. To make experiments feasible on microcontroller (MCU) hardware, a balanced subset of the dataset was used instead of the full release. Specifically, we randomly sampled 20,000 training, 4,000 validation, and 1,000 test images while enforcing a strict 1:1 ratio between human present and no human present categories in each split. The final distribution per split was as follows: 10,000/10,000 for training, 2,000/2,000 for validation, and 500/500 for testing. Random seeds were fixed to ensure reproducibility across runs. All images were resized to standard input resolutions (96×96, 160×160, and 224×224) depending on the experiment to maintain consistency and facilitate efficient preprocessing. Preprocessing was performed using TensorFlow’s API to ensure uniform image dimensions, efficient data loading, and compatibility with transfer learning pipelines. Transfer learning using pre-trained MobileNetV2 weights from ImageNet was employed to improve model generalization, reduce training time, and minimize computational overhead. This subsampling strategy preserves class balance and representativeness while enabling efficient, deployment-oriented evaluation on the target MCU platform, ensuring that results are both reproducible and practically relevant for resource-constrained Edge AI applications.

Hardware

The microcontroller Unit chosen for this research is optimized for low-power, high-performance TinyML applications. It is equipped with an ARM®Cortex®-M4 processor (64 MHz, 32-bit architecture), 1 MB of flash

memory, and 256 KB of SRAM, making it suitable for deploying compressed neural networks. Its low-power operation (3.3V) is ideal for battery-powered edge applications. The microcontroller also supports both TCP and UDP communication, enabling flexible and efficient data transmission TCP for reliable delivery and UDP for low-latency tasks. A high-resolution camera module is integrated with the microcontroller Unit to capture and process real-time images, eliminating the need for external computation. This setup facilitates energy-efficient object detection by transmitting the detection results rather than raw images, which reduces bandwidth usage. The compatibility of the microcontroller Unit with TensorFlow Lite Micro ensures seamless deployment of optimized neural network models.

Proposed framework

TensorFlow Lite Micro (<https://www.tensorflow.org/lite/microcontrollers>) for microcontrollers was used to implement MobileNetV2 and other neural network models, providing a lightweight deep learning framework optimized for resource-constrained hardware. TensorFlow Lite Micro employs quantization techniques, reducing the precision of model weights from 32-bit floating-point to 8-bit integers. This reduces model size, minimizes memory footprint, and enhances inference speed without sacrificing model performance. The framework ensures smooth deployment on low-power devices by leveraging hardware acceleration to improve computational efficiency. TensorFlow Lite Micro integration with TensorFlow APIs facilitates efficient preprocessing, model compression, and inference execution, making it an ideal choice for Edge AI applications. The framework bridges deep learning capabilities with constrained microcontroller Units, enabling efficient, scalable TinyML solutions suitable for real-world applications in IoT, smart homes, and industrial automation. To deploy a deep learning model successfully on a microcontroller Unit, we define a comprehensive, step-by-step workflow from model design to deployment. The methodology focuses on optimizing model architecture, training, compression, and deployment on resource-constrained devices, ensuring efficiency in terms of both computational resources and energy consumption.

Model architecture

For this study, MobileNetV2 was selected as the neural network architecture due to its lightweight design and efficiency in resource-constrained environments, making it well-suited for deployment on microcontroller units (MCUs) and edge devices. The architecture was further adapted to balance accuracy and computational efficiency by tuning hyperparameters, such as the width multiplier, which adjusts the total number of parameters in the model. Input image size was carefully determined, as it directly affects memory requirements and processing time: smaller image sizes reduce the memory footprint and inference latency but may slightly impact model accuracy. This configuration ensures that the model remains both computationally efficient and effective for real-time vision-based tasks.

Model training and deployment

For the Visual Wake Words (VWW) binary classification task, we adapted MobileNetV2 using transfer learning and fine-tuning. Pre-trained ImageNet weights were loaded, and the original classification head was replaced with custom layers for binary classification. To prevent overfitting, the pre-trained layers were initially frozen, training only the new classification layers for 80 epochs. Selected layers were subsequently fine-tuned at a reduced learning rate for 15 epochs, allowing the model to retain relevant features while optimizing for the specific task. Training was performed across three input resolutions (96×96 , 160×160 , and 224×224) with a batch size of 32, and the width multiplier was adjusted to control the number of convolution filters, balancing accuracy, memory usage, and inference latency. For comparison, lightweight CNN models with three convolution layers were also trained to evaluate latency and memory footprint under varying configurations.

Following training, multiple model compression strategies were applied to optimize deployment on resource-constrained microcontroller units (MCUs). Pruning was evaluated in both structured and unstructured forms to remove redundant weights and reduce computational complexity, although its effectiveness was limited by hardware constraints. Knowledge distillation (KD) was explored by transferring knowledge from MobileNetV2 to smaller CNN models, but improvements in accuracy and efficiency were minimal. Among all methods, 8-bit post-training quantization proved most effective, reducing model size by $3\text{--}4\times$ while maintaining minimal accuracy loss. The width multiplier was leveraged to further adjust model complexity, tailoring resource usage to the MCU capabilities.

The optimized models were then converted to TensorFlow Lite Micro (.tflite) format for deployment. To embed the models on the microcontroller, the .tflite files were converted into C byte arrays using tools such as `xxd`, ensuring storage within flash memory limits. The inference logic was integrated into the MCU development environment, defining input preprocessing, invoking TensorFlow Lite Micro functions, and processing outputs. Real-time performance metrics, including latency, memory utilization, and energy consumption, were measured during inference. Finally, the MCU was integrated with a Wi-Fi module supporting both TCP and UDP protocols, enabling real-time transmission of compact outputs (JSON or binary format, 1–2 KB per inference). TCP ensured reliable delivery, while UDP provided low-latency communication. Transmission metrics were collected in a line-of-sight (LOS) outdoor environment up to 150 m, including inference processing time, to assess the end-to-end feasibility of real-time object detection on resource-constrained edge devices.

Compressing model

Several model compression techniques were explored to optimize deployment on resource-constrained microcontroller Units. Knowledge distillation was first attempted by transferring knowledge from a pre-trained MobileNetV2 model to a smaller CNN model. However, this approach did not yield significant improvements in accuracy or efficiency, suggesting that future work may need to investigate custom loss functions or modify

the model's output layers to enhance the distillation process. Pruning was also evaluated in both structured and unstructured forms using TensorFlow-based implementations. While pruning can eliminate redundant parameters and reduce computational complexity, the effectiveness of this technique was limited by the hardware constraints of the selected microcontroller, indicating that further optimization at the model definition level may be necessary to achieve better outcomes. Among all methods tested, 8-bit quantization proved to be the most effective, reducing model size by a factor of three to four by converting 32-bit floating-point values into 8-bit integers. This significantly decreased memory usage and accelerated inference while maintaining minimal accuracy loss. In addition, the width multiplier parameter in MobileNetV2 was leveraged to adjust the number of convolution filters, thereby reducing model complexity and tailoring resource usage to the hardware capabilities of the microcontroller. Finally, TinyEngine was considered as a potential framework for further optimizing storage and peak memory efficiency, though its applicability was limited in this study due to compatibility issues with the selected microcontroller Unit.

Communication through TCP/UDP

After applying compression techniques, the transmission efficiency of optimized inference outputs was evaluated using both TCP and UDP protocols as shown in Fig. 1. The MCU generates compact output packets (JSON or binary format, approximately 1–2 KB per inference), which are buffered and transmitted via the Wi-Fi interface operating on the 2.4 GHz IEEE 802.11n band. Flow of inference results proceeds from the MCU to the host, where TCP ensures reliable delivery with acknowledgments and retransmissions, while UDP enables lower-latency, best-effort delivery without handshake overhead. This dual-protocol support enhances adaptability in resource-constrained environments where both reliability and real-time responsiveness are critical. To provide a comprehensive evaluation, tests were conducted in an outdoor line-of-sight (LOS) environment between the microcontroller and the host device, with a maximum tested range of 150 m. The reported latency values, which include inference processing time on the MCU, represent one-way transmission from the microcontroller to the host. On reception, the host device logs and analyzes the packets, validating end-to-end trade-offs in latency, reliability, and energy efficiency.

System architecture

The proposed system architecture for MCU-based real-time object detection is illustrated in Fig. 2. The design emphasizes energy efficiency, low-latency inference, and scalable deployment for resource-constrained edge environments. The architecture consists of three primary components: the input capture module, the on-device inference engine, and the communication interface.

Input Capture Module: Image data is captured in real-time using a high-resolution camera integrated directly with the MCU. The captured frames are temporarily buffered in the MCU's SRAM to allow preprocessing without relying on external memory. Preprocessing steps include resizing, normalization, and optional RGB channel separation, ensuring that the input images match the dimensions required by the trained CNN models (e.g., 96×96 , 160×160 , or 224×224). This module minimizes the need for transmitting raw images, significantly reducing bandwidth usage and energy consumption.

On-Device Inference Engine: The quantized MobileNetV2 model, along with other comparative models, is deployed on the ARM Cortex-M4 MCU using TensorFlow Lite Micro. The inference engine handles all computationally intensive operations locally, including convolutions, activations, and pooling layers. Model execution leverages optimized routines provided by TensorFlow Lite Micro, enabling efficient use of CPU cycles and SRAM. Only the inference results typically labels and confidence scores are retained for transmission, ensuring minimal data overhead. This design allows the MCU to perform autonomous, real-time object detection without reliance on cloud computing resources, which is critical for applications requiring low latency and high reliability.

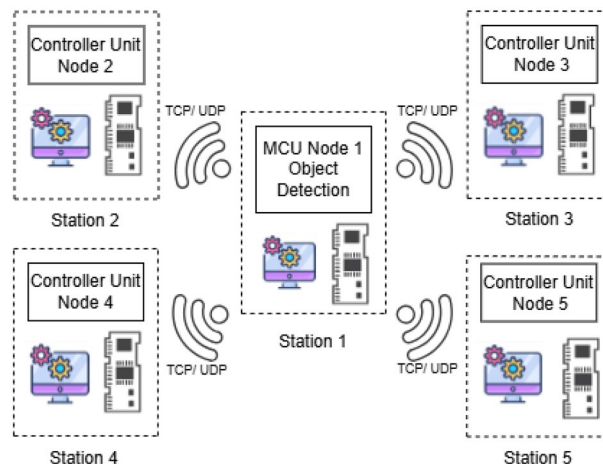


Fig. 1. Integration of TCP/UDP for Data Transmission.

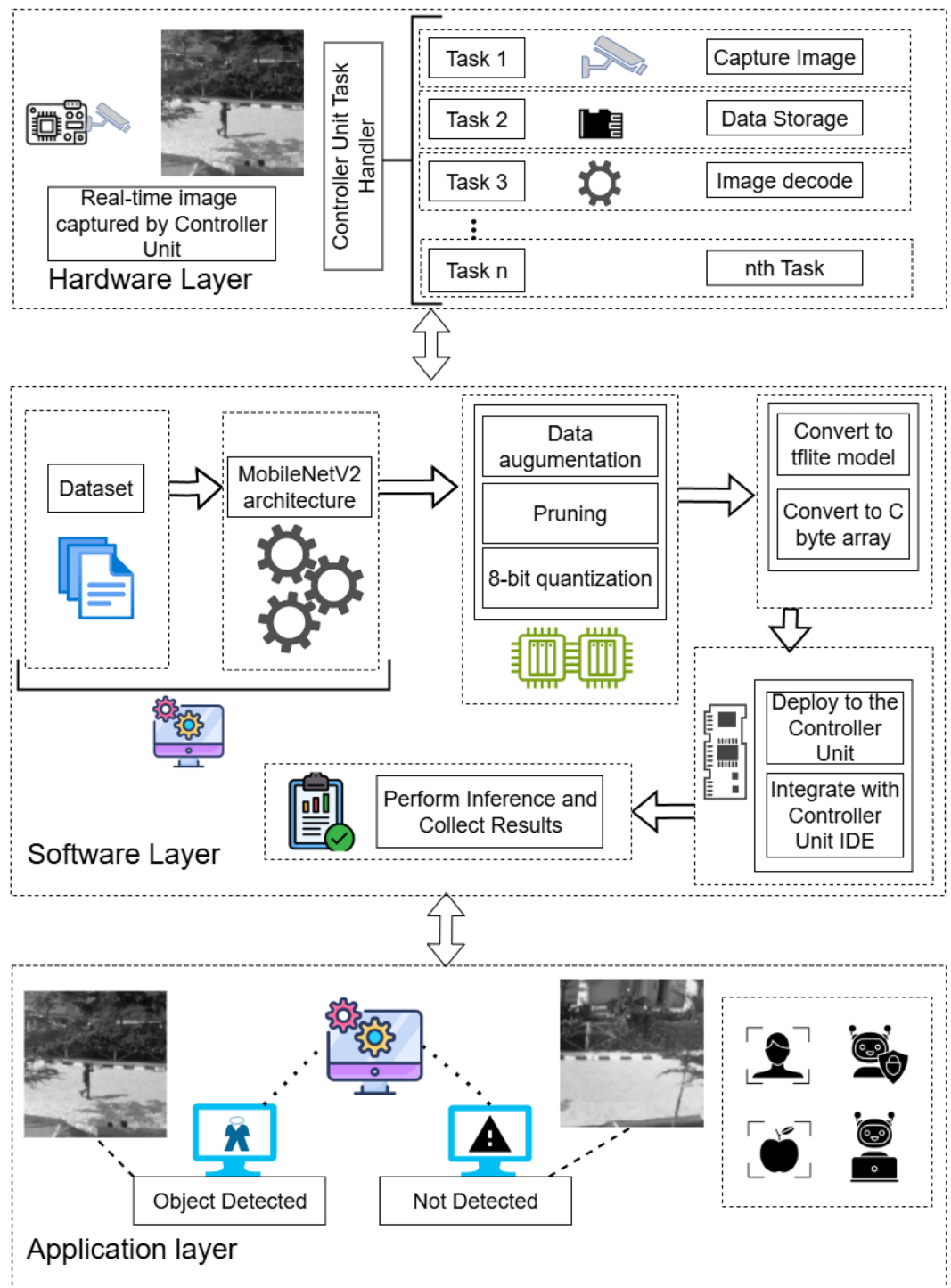


Fig. 2. System Architecture for Object Detection.

Communication Interface: The processed outputs are transmitted to a host device via a Wi-Fi module integrated with the MCU. The system supports both TCP and UDP protocols, providing flexibility depending on application requirements. TCP is used when reliable delivery is paramount, incorporating acknowledgments and retransmissions to ensure data integrity. UDP is used for low-latency transmission, suitable for real-time monitoring scenarios where occasional packet loss is tolerable. Each inference output is encapsulated in compact packets (1–2 KB), either in JSON or binary format, and buffered before transmission to optimize throughput and reduce transmission energy.

Host Device Integration: On the receiving end, a host device logs incoming inference results, visualizes detections, and optionally triggers downstream actions such as alerts or control commands. The host device also

facilitates evaluation of communication performance, measuring metrics like end-to-end latency, packet loss, and energy consumption under different protocol configurations.

System-Level Considerations: The architecture emphasizes energy efficiency and resource optimization at each stage. By performing inference on-device, the system reduces reliance on network transmission, thereby saving bandwidth and energy. Dual-protocol communication allows dynamic adaptation to network conditions: UDP for time-critical tasks and TCP for reliability-sensitive applications. Memory usage and flash constraints are explicitly managed, with quantization and width-multiplier adjustments ensuring models fit within the 256 KB SRAM and 1 MB flash of the MCU.

Performance evaluation

The performance of the proposed system was evaluated through a series of experiments conducted both on a laptop and on the microcontroller Unit. On the laptop, we first assessed storage efficiency by comparing network architectures before and after quantization, which revealed substantial reductions in model size and confirmed the suitability of compression for memory-constrained environments. We further examined the impact of batch size, input image dimensions, and network architecture on inference latency, highlighting the computational trade-offs inherent in CNN models. Quantization was additionally tested across different image resolutions and architectures, and the results demonstrated consistent improvements in inference speed, providing important insights into optimizing performance under strict storage and latency constraints. On the microcontroller Unit, experiments focused on determining the maximum deployable model size by evaluating various neural network models in terms of their memory footprint in kilobytes. Energy consumption was measured during inference using a multimeter to capture real-time power profiles, while compilation time and potential memory overflow were analyzed relative to model size to ensure efficient deployment within the hardware's limitations. Finally, communication performance was evaluated by integrating both TCP and UDP protocols and testing them under varying network conditions, including high traffic loads and intermittent connectivity. Latency and energy consumption were recorded for both protocols, revealing that TCP provided higher reliability through acknowledgments and retransmissions, while UDP achieved significantly lower latency due to its lightweight communication mechanism. These results confirmed that a dual-protocol approach offers flexibility for Edge AI deployments, enabling the system to switch between reliability and real-time responsiveness depending on application requirements.

Results

Model compression and real-time image analysis

In this study, we aimed to evaluate the performance of model compression techniques, specifically quantization, to reduce the storage footprint of Convolutional Neural Networks (CNNs)⁴² while maintaining their effectiveness for deployment in resource-constrained Edge AI environments. Figure 3 clearly demonstrates the efficiency of quantization, which reduces the model size by approximately 3–4× compared to the original, uncompressed versions. This reduction is critical for ensuring that deep learning models can fit within the storage limits of embedded systems like microcontrollers, which typically have Flash memory capped at 1 MB or less. The trade-off here is that while quantization reduces model size and memory usage, it also leads to a slight degradation in performance. However, this degradation is minimal and acceptable, especially when considering the substantial reduction in storage requirements and the potential for real-time, low-latency inference.

To assess the impact of model compression on real-time image analysis, we captured live images and processed them through the CNN. As shown in Figure 4, we decomposed the captured images into Red, Green, and Blue (RGB) channels, each visualized in grayscale to reveal variations in intensity. This decomposition provides

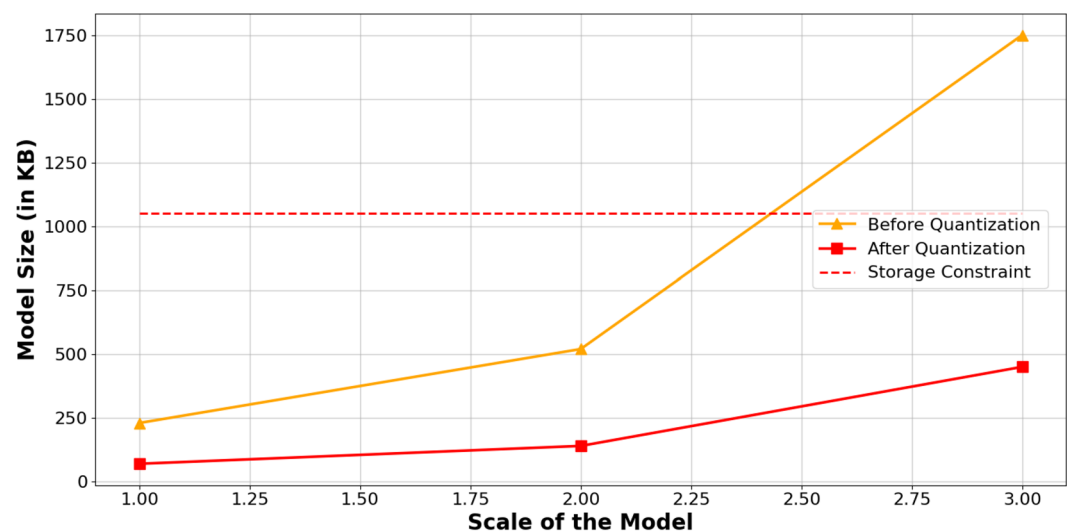


Fig. 3. Storage footprint of different architectures before and after quantization with the 1 MB constraint.

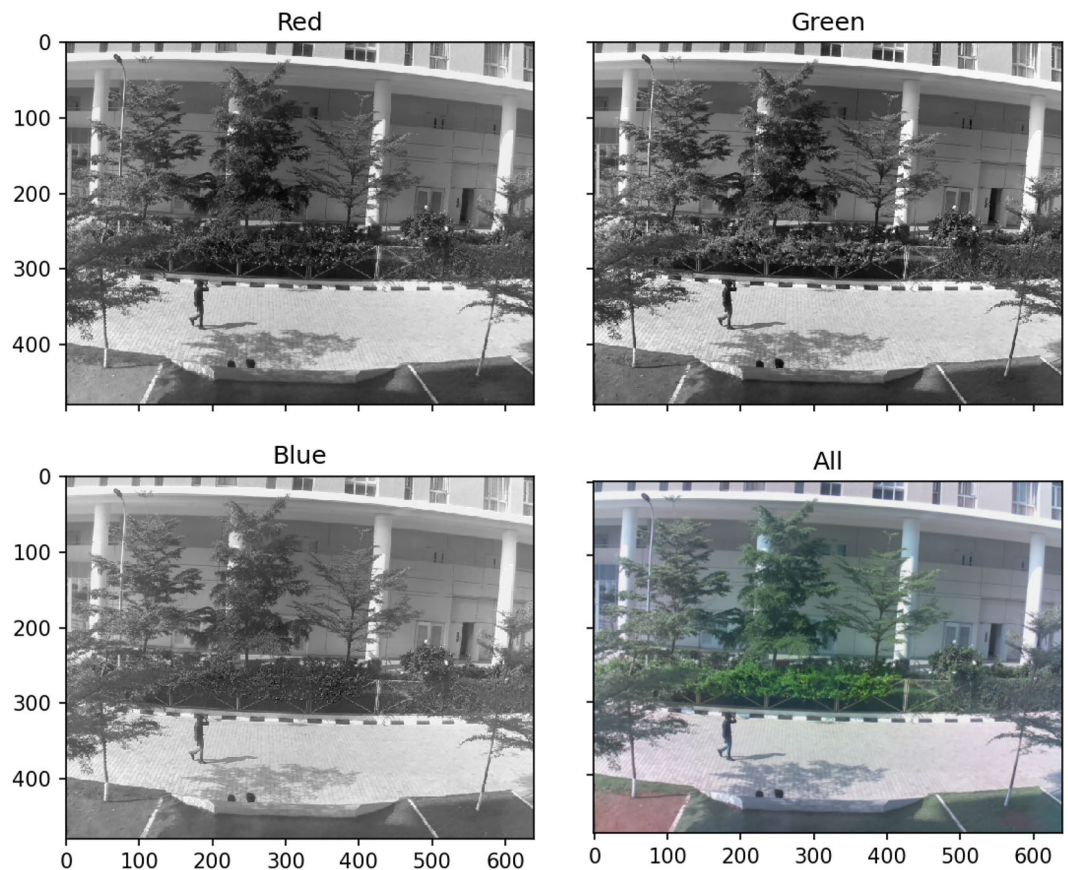


Fig. 4. Real-time image and RGB channel decomposition used for preprocessing validation.

insight into how the CNN captures and processes color information at different stages of image processing, which is vital for tasks such as object detection and classification. The reconstructed “All” image, created by merging the RGB channels, demonstrates the model’s ability to preserve essential features during preprocessing. The strength of this approach lies in its ability to process real-time data efficiently, even on memory-constrained devices, by utilizing optimized models that maintain critical features. However, the trade-off is that while model size is reduced, the preprocessing steps may become slightly more complex, and fine-tuning is required to avoid losing meaningful data from the image. Nevertheless, this approach is highly beneficial for Edge AI applications, where efficiency and real-time processing are paramount.

Effect of batch size on inference latency

The inference latency of CNN models is significantly affected by the batch size, with larger batch sizes generally resulting in faster processing due to parallelism in computation. As shown in Figure 5, when the batch size is set to 1, the model experiences the highest latency because each input is processed sequentially. This is a well-known issue in Edge AI, where small batch sizes expose the overhead of memory transfers and kernel launches, making the inference process slower. However, increasing the batch size allows the model to process multiple inputs simultaneously, reducing the latency by enabling better utilization of the hardware’s computational resources.

Despite the advantages of increasing the batch size, Fig. 5 also shows that the reduction in latency plateaus beyond a batch size of 16. This saturation occurs because the system’s memory resources, such as caches and memory bandwidth, become saturated. Larger batch sizes place higher demands on these resources, leading to memory bottlenecks. The trade-off here is that while increasing the batch size beyond 16 can still provide parallelism benefits, the system might reach a point of diminishing returns, especially on embedded devices with limited memory capacity.

Additionally, the impact of quantization on latency is examined in Fig. 6. The results show that quantized models consistently exhibit lower latency than their non-quantized counterparts. This improvement is more pronounced in larger models and higher resolutions, where the computational cost of convolutional operations is higher. Quantization reduces the precision of model weights, which decreases the number of operations required for each inference, thus improving computational efficiency. This makes quantization an effective tool for reducing latency without significantly compromising accuracy. The strength of quantization lies in its ability to accelerate inference, especially in real-time applications where low latency is critical. However, the trade-off is that quantization introduces some accuracy loss, particularly for smaller models where the precision of the weights has a more noticeable impact.

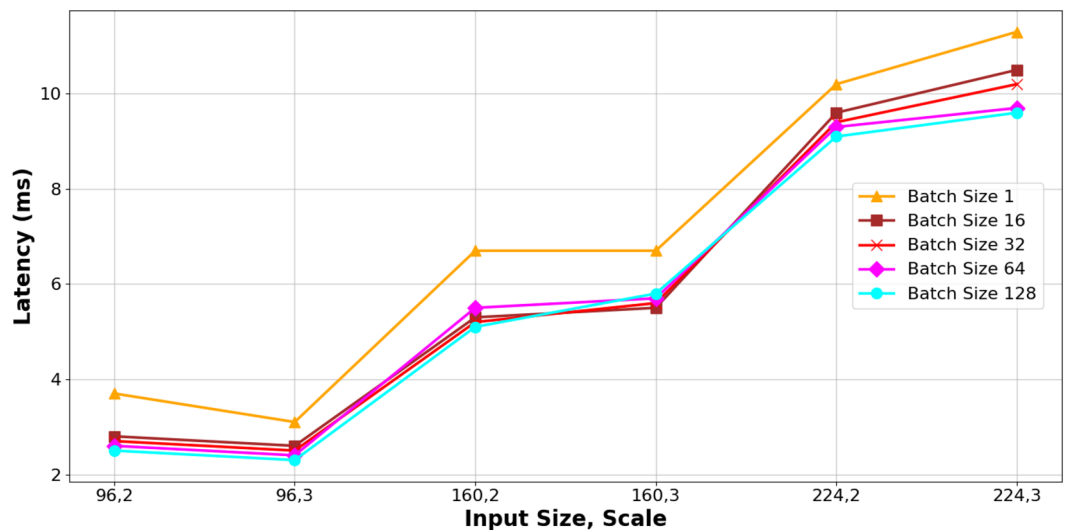


Fig. 5. Effect of batch size, input resolution, and network scale on inference latency.

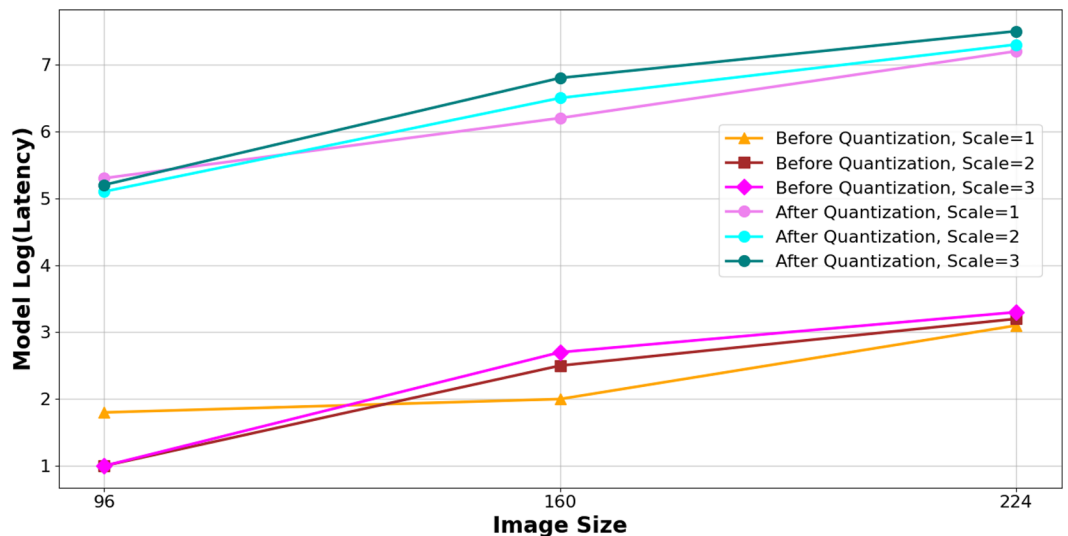


Fig. 6. Latency comparison across image resolutions and network scales before and after quantization.

CNN latency vs. scale and quantization

In Fig. 6, we explore the relationship between model scale, input resolution, and quantization on latency. As expected, larger models, which have more convolutional layers and parameters, exhibit higher latency due to the increased number of computations required. This increase in latency is particularly significant when processing larger images, such as those with a resolution of 224×224 . Larger models not only require more computations but also use more memory bandwidth, which further exacerbates the latency issue.

Quantization helps mitigate this problem by reducing the model's precision from 32-bit floating-point to 8-bit integers. This reduction in precision significantly decreases the number of computations needed during inference, thereby reducing latency. The benefit of quantization becomes more apparent with larger models and higher input resolutions, where the computational burden is heavier. However, the trade-off is that applying quantization to very small models or lower resolutions may not provide substantial improvements in latency, as the operations required for smaller inputs are already relatively few.

In summary, quantization is a valuable technique for mitigating latency in larger models, particularly when deployed in real-time, resource-constrained environments. It effectively reduces the computational cost and memory bandwidth requirements, but its benefits are more pronounced in larger, more complex models and higher-resolution inputs.

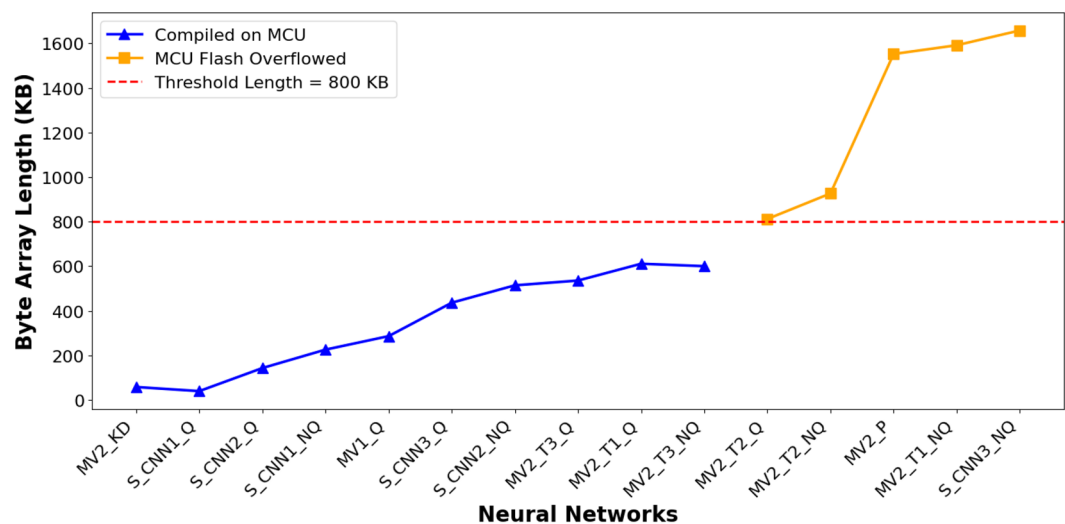


Fig. 7. Byte array sizes of models, highlighting feasibility under the 800 KB MCU deployment limit.

Model	Byte array (KB)	Accuracy	Compile (ms)	Energy (J)	Latency (ms)
MV2_KD	58.36	0.59	–	–	–
S_CNN1_Q	39.99	0.58	113000	14.3	8364
S_CNN2_Q	143.45	0.66	108000	10.6	14536
S_CNN1_NQ	226.06	0.58	116000	14.8	7234
MV1_Q	286.54	0.86	166000	22.1	3472
S_CNN3_Q	436.48	0.69	114000	14.9	9358
S_CNN2_NQ	514.40	0.66	111000	14.8	8974
MV2_T3_Q	535.90	0.88	114000	15.3	13478
MV2_T1_Q	611.22	0.84	133000	15.4	14983
MV2_T3_NQ	600.42	0.89	217000	17.3	4793
MV2_T2_Q	812.01	0.87	–	–	–
MV2_T2_NQ	926.52	0.87	–	–	–
MV2_P	1552.03	0.77	–	–	–
MV2_T1_NQ	1591.28	0.85	–	–	–
S_CNN3_NQ	1657.67	0.69	–	–	–

Table 3. Summary of model footprints and on-device measurements.

Storage footprint across architectures

For embedded systems, storage is one of the most significant constraints when deploying deep learning models. As shown in Fig. 3, storage efficiency is paramount, especially when Flash memory is limited to around 1 MB or less. Without quantization, many models exceed the storage limit, making them unsuitable for deployment on microcontrollers. However, applying quantization significantly reduces the storage size, as seen in Fig. 3, where quantized models fall well within the 1 MB storage constraint.

The strength of quantization lies in its ability to compress the model size without sacrificing much in terms of performance, making it possible to deploy larger, more complex models on microcontrollers. However, the trade-off is that compression techniques such as quantization might introduce slight accuracy loss, especially when working with very small models or models that already have limited capacity. In practice, quantization ensures that models with high accuracy, such as MV1_Q (286.54 KB) and S_CNN3_Q (436.48 KB), can be deployed effectively, while models like MV2_P (1,552.03 KB) exceed the memory limit and are unsuitable for deployment without further compression.

Byte array, compilation time, energy, and accuracy

Figure 7 provides an overview of the byte array sizes for different models, with those exceeding 800 KB marked as infeasible for deployment. Models like MV2_T2_Q (812.01 KB) and MV2_T2_NQ (926.52 KB) are larger than the permissible byte array size for the microcontroller Unit, making them unsuitable for deployment in their current form. This observation is further detailed in Table 3, which summarizes the footprints and on-

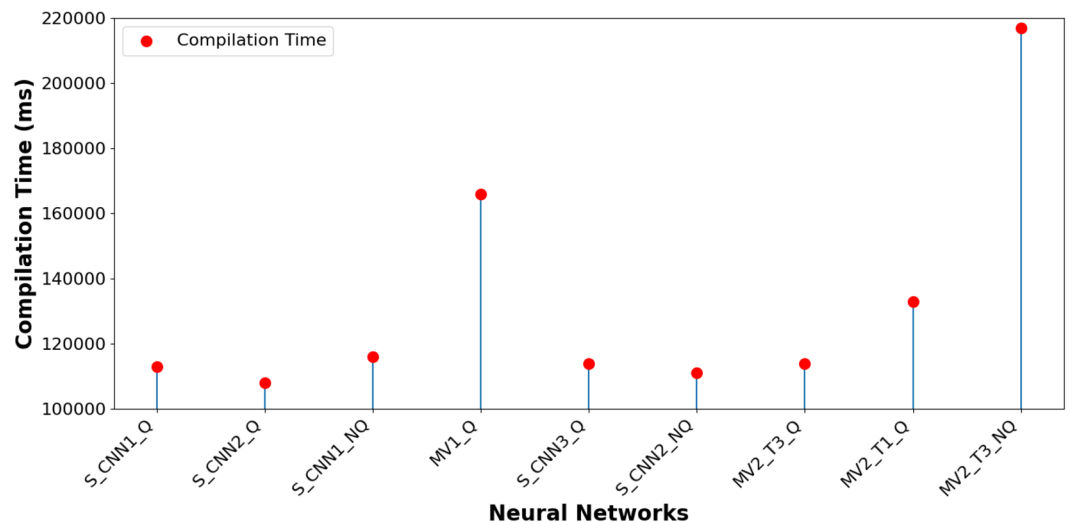


Fig. 8. Compilation time of different models on the MCU toolchain.

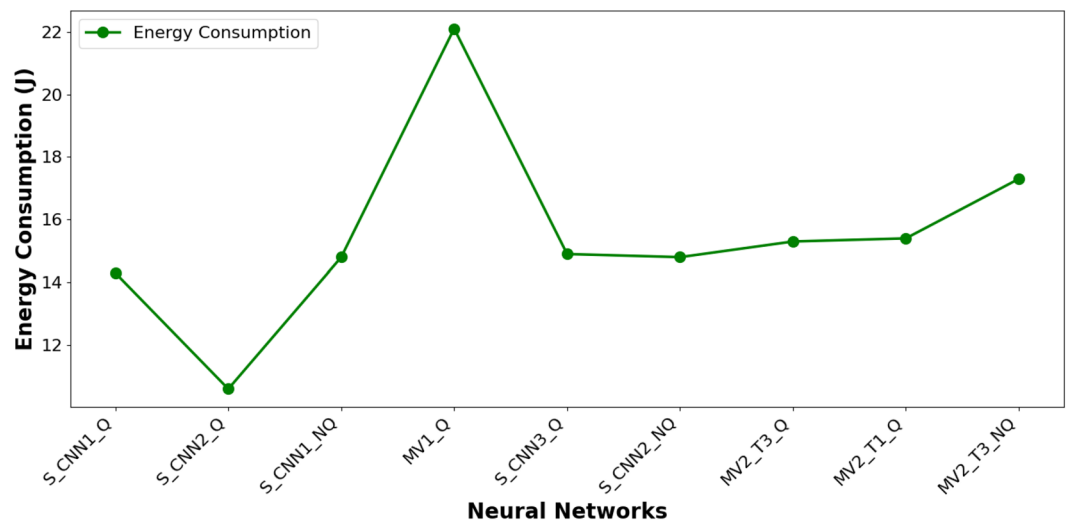


Fig. 9. Energy per inference measured on the microcontroller Unit.

device measurements across all evaluated models. This is a critical observation for deployment in real-world systems, where model size must be optimized for both storage and memory efficiency.

Compilation time, shown in Fig. 8, reflects the complexity of the model and the toolchain used for compilation. The results show a non-monotonic relationship between compilation time and model size, suggesting that other factors, such as memory optimization and kernel fusion, play a role in determining the time required for compilation. Despite the variations in compilation time, energy consumption remains consistent across models with similar architectures, as shown in Fig. 9. Energy consumption correlates with computational intensity models requiring more operations consume more energy. However, the overall energy usage remains within reasonable limits, making it feasible to deploy these models in battery-powered Edge AI applications.

Accuracy, shown in Fig. 10, varies across models, with **MV2_KD** (58.36 KB) demonstrating the lowest accuracy at 0.59. This model is highly compact but sacrifices accuracy in favor of a reduced footprint. On the other hand, larger models such as **MV1_Q** (286.54 KB) and **MV2_T3_Q** (535.90 KB) achieve high accuracy (0.86–0.88) while remaining within the storage constraints. The trade-off here is that smaller models may offer advantages in terms of storage and memory usage but could suffer from reduced performance, especially when accuracy is critical for the task at hand.

Time and energy analysis on the microcontroller unit

Compilation time and energy consumption measurements provide valuable insights into how hardware limitations affect the deployment of deep learning models. Figure 8 shows that compilation time does not strictly correlate with model size. Smaller models like **S_CNN1_Q** (39.99 KB) take longer to compile compared to larger

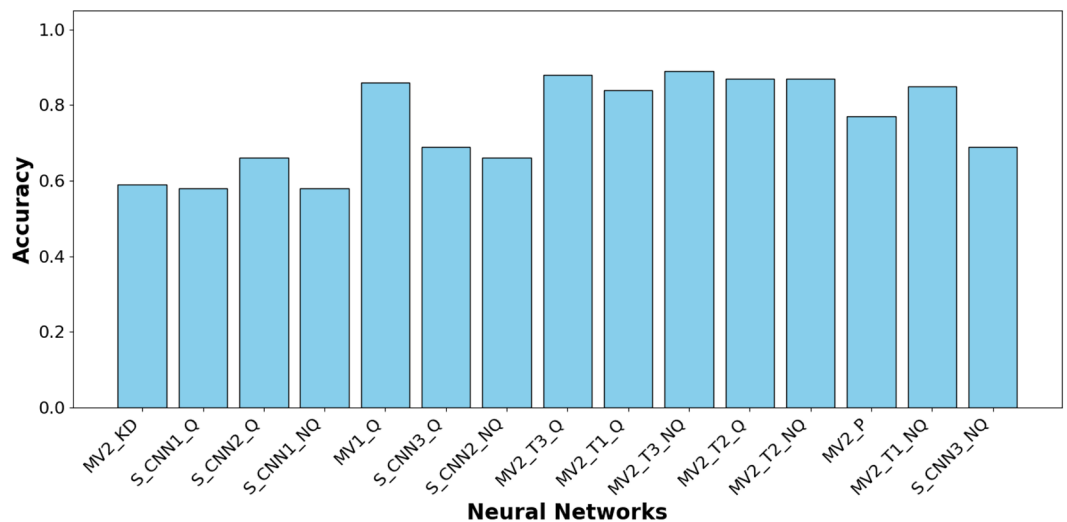


Fig. 10. Accuracy comparison of quantized and non-quantized models across different architectures.

models like S_CNN2_Q (143.45 KB). This highlights that factors such as memory optimization and toolchain behavior contribute to the compilation speed, not just the byte size.

Energy consumption, as shown in Fig. 9, correlates well with model complexity. Larger models consume more energy per inference due to the greater number of operations and higher memory demands. However, the energy consumption remains within the acceptable range for Edge AI applications, making it feasible to deploy these models on battery-powered devices with careful energy management strategies. A comprehensive view of these trade-offs is presented in Table 3.

Wireless performance over TCP/UDP

For real-world communication scenarios, latency and energy consumption were measured over TCP and UDP protocols. As shown in Fig. 1 (not shown here), UDP provides significantly lower latency compared to TCP. The reduction in latency is primarily due to the minimal protocol overhead in UDP, which is ideal for time-sensitive applications. However, TCP provides reliable data transmission at the cost of higher latency, which may be more suitable for scenarios where data integrity is more important than speed. Energy consumption during communication remains low for both protocols, confirming the feasibility of deploying these models in wireless, battery-powered Edge AI systems.

Discussion

This study demonstrates the feasibility of deploying deep learning models on low-power MCUs for real-time object detection, while systematically analyzing the trade-offs between model accuracy, storage footprint, compilation time, energy consumption, and latency. The experimental results highlight that quantization is the most effective and practical model compression technique, consistently reducing model size by a factor of 3–4× and enabling deployment within the strict 800 KB memory budget of the MCU. For instance, quantized MobileNetV1 (MV1_Q, 286.54 KB) and MobileNetV2-T3 (MV2_T3_Q, 535.90 KB) achieved accuracies of 0.86 and 0.88, respectively, while remaining feasible for on-device inference. In contrast, larger models such as MV2_P (1552.03 KB) or MV2_T1_NQ (1591.28 KB) exceeded the available memory, underscoring the necessity of compression for MCU-based deployments.

Pruning was explored as a potential compression technique, but the resulting models still exceeded the MCU's flash/DRAM budget and therefore could not be deployed. Although pruning reduced parameter counts at the training stage, these reductions were insufficient to bring the models within the 800 KB deployment threshold, limiting their utility in this context. Knowledge distillation (KD) produced highly compact models such as MV2_KD (58.36 KB), but this came at the cost of accuracy (0.59), rendering them unsuitable for reliable real-time object detection. These findings confirm that, among the tested strategies, quantization is the only approach that consistently enables successful deployment on MCUs without severely compromising performance.

The results also highlight important trade-offs between accuracy, resource utilization, and energy efficiency. Lightweight models such as S_CNN1_Q (39.99 KB) offered extremely low memory footprints but achieved only 0.58 accuracy, limiting their applicability in accuracy-sensitive scenarios. On the other hand, more complex models like MV2_T3_NQ (600.42 KB, 0.89 accuracy) delivered strong performance but required substantially higher compilation times and energy consumption. This tension illustrates a key design consideration in TinyML deployments: smaller models provide efficiency but may compromise accuracy, while larger models improve accuracy at the expense of resource usage and inference cost.

Communication analysis further demonstrated that the choice of transmission protocol directly affects system-level performance. UDP consistently provided lower latency due to its lightweight, best-effort design, making it well suited for time-sensitive inference tasks. In contrast, TCP introduced additional delay from

acknowledgments and retransmissions but ensured reliable data delivery, which is critical in applications where integrity cannot be compromised. These results suggest that adaptive protocol selection, depending on task requirements, can further improve the robustness and responsiveness of MCU-based Edge AI deployments.

Overall, the study confirms that quantization-enabled MobileNet variants offer the best trade-offs for MCU deployment, achieving accuracy above 0.85 while staying within strict memory, energy, and latency constraints. While pruning and KD were investigated, their limitations in terms of feasibility and accuracy restricted their applicability in this setup. Future work should explore hybrid compression approaches that combine quantization with hardware-aware pruning or improved distillation strategies, as well as the use of specialized MCU accelerators to further enhance inference speed and energy efficiency. Addressing these challenges will strengthen the deployment of real-time, energy-efficient object detection systems in Edge AI and IoT applications.

Conclusion

This work presented a real-time, energy-efficient object detection system deployed on a low-power microcontroller unit (MCU). By systematically evaluating multiple compression techniques, it was shown that quantization is the only strategy that consistently enables deployment under the strict 800 KB memory budget while maintaining acceptable accuracy. Quantized MobileNet variants such as MV1_Q and MV2_T3_Q achieved accuracies above 0.85 with manageable latency and energy consumption, demonstrating their suitability for constrained embedded platforms. In contrast, pruning reduced parameter counts but the resulting models remained too large to fit within MCU memory, and knowledge distillation (KD) produced compact models at the expense of significant accuracy degradation. Communication experiments further revealed that protocol choice impacts end-to-end performance, with UDP offering lower latency and TCP ensuring reliable transmission. Collectively, these results validate that quantization-enabled MobileNet architectures provide the most effective balance between accuracy, efficiency, and deployability for TinyML-based real-time inference on MCUs.

Future work

While quantization emerged as the most practical approach in this study, several open challenges remain for future exploration. Pruning requires deeper hardware-aware adaptation to ensure models not only reduce parameter counts but also fit within strict MCU memory budgets. Knowledge distillation techniques must be refined, for instance through tailored loss functions or intermediate feature alignment, to improve accuracy in compact student models without significantly increasing their footprint. Hybrid strategies that integrate quantization with structured pruning or KD may also provide better trade-offs between accuracy and efficiency. Moreover, experimental evaluation of security and network stability aspects including packet loss, jitter, and protocol-level overheads in TCP/UDP communication remains crucial for ensuring reliable and secure operation in real-world deployments. Finally, leveraging specialized MCU accelerators, such as DSP co-processors or energy-aware neural execution engines, presents a promising avenue to further reduce inference latency and energy consumption. Collectively, these directions will contribute to establishing more robust, efficient, and scalable TinyML solutions for smart IoT, industrial monitoring, and real-time environmental sensing applications.

Data availability

The Visual Wake Words (VWW) dataset employed in this study is derived from the COCO dataset and is publicly available for research purposes. It can be generated by filtering COCO annotations to construct a binary classification task (“person” vs. “not-person”). Detailed instructions and scripts for generating the dataset, along with preprocessing utilities, are openly accessible

Received: 19 July 2025; Accepted: 6 November 2025

Published online: 05 December 2025

References

- Mittal, P. A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artif. Intell. Rev.* **57**, 242 (2024).
- Hua, H. et al. Edge computing with artificial intelligence: A machine learning perspective. *ACM Comput. Surv.* **55**, 1–35. <https://doi.org/10.1145/3555802> (2023).
- Chen, J. & Ran, X. Deep learning with edge computing: A review. *Proc. IEEE* **107**, 1655–1674 (2019).
- Soni, D. & Kumar, N. Machine learning techniques in emerging cloud computing integrated paradigms: A survey and taxonomy. *J. Netw. Comput. Appl.* **205**, 103419 (2022).
- Iqbal, S. M. A., Mahgoub, I., Du, E., Leavitt, M. A. & Asghar, W. Advances in healthcare wearable devices. *npj Flex. Electron.* **5**, 9. <https://doi.org/10.1038/s41528-021-00107-x> (2021).
- Zhou, Y., Xia, L., Zhao, J., Yao, R. & Liu, B. Efficient convolutional neural networks and network compression methods for object detection: a survey. *Multimed. Tools Appl.* **83**, 10167–10209 (2024).
- Msuya, H. G. & Maiseli, B. J. Deep learning model compression techniques: Advances, opportunities, and perspective. *Tanzania J. Eng. Technol.* **42**, 65–83. <https://doi.org/10.52339/tjet.v42i2.853> (2023).
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. & Chen, L.-C. MobileNetV2: Inverted residuals and linear bottlenecks (2018). [Arxiv:1801.04381](https://arxiv.org/abs/1801.04381).
- Dong, K., Zhou, C., Ruan, Y. & Li, Y. Mobilenetv2 model for image classification. In *2020 2nd International Conference on Information Technology and Computer Application (ITCA)*, 476–480, <https://doi.org/10.1109/ITCA52113.2020.00106> (2020).
- Shen, H., Wang, Z., Zhang, J. & Zhang, M. L-net: A lightweight convolutional neural network for devices with low computing power. *Inf. Sci.* **660**, 120131 (2024).
- Agarwal, P. & Alam, M. A lightweight deep learning model for human activity recognition on edge devices. *Procedia Comput. Sci.* **167**, 2364–2373. <https://doi.org/10.1016/j.procs.2020.03.289>. (International Conference on Computational Intelligence and Data Science, 2020)
- Gholami, A. et al. A survey of quantization methods for efficient neural network inference (2021). [Arxiv:2103.13630](https://arxiv.org/abs/2103.13630).
- Banner, R., Hubara, I., Hoffer, E. & Soudry, D. Scalable methods for 8-bit training of neural networks (2018). [Arxiv:1805.11046](https://arxiv.org/abs/1805.11046).

14. Chmiel, B. et al. Neural gradients are near-lognormal: Improved quantized and sparse training (2020). [Arxiv:2006.08173](#).
15. Haryono, W. Comparative analysis of tcp and udp protocol performance in sending text messages using chatting applications. *J. Inotera* **10**, 41–48 (2025).
16. Bansal, S., Shorey, R. & Kherani, A. Performance of tcp and udp protocols in multi-hop multi-rate wireless networks. In *2004 IEEE Wireless Communications and Networking Conference (IEEE Cat. No.04TH8733)*, 1, 231–236. <https://doi.org/10.1109/WCNC.2004.1311548> (2004).
17. Odema, M., Chen, L., Levorato, M. & Al Faruque, M. A. Testudo: Collaborative intelligence for latency-critical autonomous systems. *IEEE Transactions on Comput. Des. Integr. Circuits Syst.* (2022).
18. Hinton, G., Vinyals, O. & Dean, J. Distilling the knowledge in a neural network (2015). [Arxiv:1503.02531](#).
19. Polino, A., Pascanu, R. & Alistarh, D. Model compression via distillation and quantization (2018). [Arxiv:1802.05668](#).
20. Phuong, M. & Lampert, C. Towards understanding knowledge distillation. In Chaudhuri, K. & Salakhutdinov, R. (eds.) *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, 5142–5151 (PMLR, 2019).
21. Koonce, B. *MobileNetV3*, 125–144 (Apress, 2021).
22. Howard, A. et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (2019).
23. Krivokapic, B., Tomovic, S., Radusinovic, I. & Jovanovic, A. Implementation and performance evaluation of convolutional neural network models for low-power microcontrollers with constrained resources. In *2024 28th International Conference on Information Technology (IT)*, 1–4, <https://doi.org/10.1109/IT61232.2024.10475765> (2024).
24. Pacheco, R. G., Couto, R. S. & Simeone, O. On the impact of deep neural network calibration on adaptive edge offloading for image classification. *J. Netw. Comput. Appl.* **217**, 103679 (2023).
25. Vidal, C., Malysz, P., Naguib, M., Emadi, A. & Kollmeyer, P. J. Estimating battery state of charge using recurrent and non-recurrent neural networks. *J. Energy Storage* **47**, 103660. <https://doi.org/10.1016/j.est.2021.103660> (2022).
26. Banbury, C. et al. Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers. *Proc. Mach. Learn. Syst.* **3**, 517–532 (2021).
27. Chowdhery, A., Warden, P., Shlens, J., Howard, A. & Rhodes, R. Visual wake words dataset. arXiv preprint [arXiv:1906.05721](#) (2019).
28. Howard, A. G. et al. MobileNets: Efficient convolutional neural networks for mobile vision applications (2017). [Arxiv:1704.04861](#).
29. Zhou, Y., Chen, S., Wang, Y. & Huan, W. Review of research on lightweight convolutional neural networks. In *2020 IEEE 5th Information Technology and Mechatronics Engineering Conference (ITOEC)*, 1713–1720, <https://doi.org/10.1109/ITOEC49072.2020.9141847> (2020).
30. Deng, J. et al. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255 (IEEE, 2009).
31. Krizhevsky, A., Sutskever, I. & Hinton, G. E. Imagenet classification with deep convolutional neural networks. In Pereira, F., Burges, C., Bottou, L. & Weinberger, K. (eds.) *Advances in Neural Information Processing Systems*, vol. 25 (Curran Associates, Inc., 2012).
32. Moosmann, J., Giordano, M., Vogt, C. & Magno, M. Tinyssimoyolo: A quantized, low-memory footprint, tinyml object detection network for low power microcontrollers. In *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 1–5 (IEEE, 2023).
33. Howard, A. et al. Searching for MobileNetV3 (2019). [Arxiv:1905.02244](#).
34. Dai, J. Real-time and accurate object detection on edge device with tensorflow lite. *J. Physics: Conf. Ser.* **1651**, 012114 (2020).
35. Lokhande, H. & Ganorkar, S. R. Object detection in video surveillance using MobileNetV2 on resource-constrained low-power edge devices. *Bull. Electr. Eng. Informatics* **14**, 357–365 (2025).
36. Jaiswal, T., Pandey, M. & Tripathi, P. Real-time multiple object detection using raspberry pi and tiny-ml approach. *Recent Adv. Electr. & Electron. Eng.* **18**, 244–255 (2025).
37. Stojanović, D., Sentić, S., Stojanović, N. & Stamenković, T. Resource-aware object detection and recognition using edge ai across the edge-fog computing continuum. *Comput. Sci. Inf. Syst.* 20–20 (2025).
38. El Zeinaty, C., Hamidouche, W. & Herrou, G. & Menard, D. Foundations, comparative analysis, challenges, and emerging solutions. *ACM Comput. Surv.* (2025).
39. Novac, P.-E., Boukli Hacene, G., Pegatoquet, A., Miramond, B. & Gripon, V. Quantization and deployment of deep neural networks on microcontrollers. *Sensors* **21**, 2984 (2021).
40. Liberis, E. & Lane, N. D. Differentiable neural network pruning to enable smart applications on microcontrollers. *Proc. ACM on Interactive, Mobile, Wearable and Ubiquitous Technol.* **6**, 1–19 (2023).
41. Warden, P. & Situnayake, D. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (O'Reilly Media, 2019).
42. O'Shea, K. & Nash, R. An introduction to convolutional neural networks. *CoRR* **abs/1511.08458** (2015). [Arxiv:1511.08458](#).

Acknowledgements

The authors would like to acknowledge the support of the Futuristic Ubiquitous Networking (FUN) LAB and the research facilities at SRM University for providing the seed grant SRMAP/URG/SEED/2023-24/036, which made this research possible.

Author contributions

C.M.B. conceived and designed the experiment, conducted the performance evaluations, and analyzed the results. P.K. and N.P. were responsible for the development and optimization of the neural network model, and they implemented the compression and quantization techniques. F.G. supervised the hardware configuration, system integration, and communication protocol setup. M.M.H. supervised the neural network design, model training, and optimization. M.A. contributed to data analysis and the implementation of TCP/UDP protocols for data transmission. A.A.D. assisted with the integration of energy-efficient techniques and performance evaluation. J.F. contributed to the hardware setup and provided critical insights into the communication protocols. All authors contributed to writing and reviewing the manuscript.

Funding

The authors declare that no funding was received for this research. The work was carried out using internal resources from the authors' affiliated institution, SRM University AP, and did not involve any external financial support or grants.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to J.F.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025